

Assignment 2

brandon rumbarger

2026-02-18

```
library(dplyr)
```

```
## Warning: package 'dplyr' was built under R version 4.5.2
```

```
library(ggplot2)
library(caret)
library(class)
```

```
bank <- read.csv("C:\\Users\\brumb\\OneDrive\\Documents\\R\\UniversalBank-1.csv")
```

```
bank <- bank %>%
  select(-ID, -ZIP.Code)
```

```
bank <- bank %>%
  mutate(
    Personal.Loan = factor(Personal.Loan),
    Education = factor(Education),
    Securities.Account = factor(Securities.Account),
    CD.Account = factor(CD.Account),
    Online = factor(Online),
    CreditCard = factor(CreditCard)
  )
```

```
dummies <- dummyVars(~ ., data = bank %>% select(-Personal.Loan))
```

```
bank_dummy <- data.frame(
  predict(dummies, newdata = bank %>% select(-Personal.Loan))
)
```

```
bank_dummy$Personal.Loan <- bank$Personal.Loan
```

```
preProc <- preProcess(bank_dummy[, -ncol(bank_dummy)],
  method = c("center", "scale"))
```

```
bank_norm <- predict(preProc, bank_dummy[, -ncol(bank_dummy)])
bank_norm$Personal.Loan <- bank_dummy$Personal.Loan
```

```

set.seed(1)

trainIndex <- createDataPartition(bank_norm$Personal.Loan,
                                   p = 0.6, list = FALSE)

train <- bank_norm[trainIndex, ]
valid <- bank_norm[-trainIndex, ]

```

Customer profile:

- Age = 40
- Experience = 10
- Income = 84
- Family = 2
- CCAvg = 2
- Education_2 = 1
- Mortgage = 0
- Securities Account = 0
- CD Account = 0
- Online = 1
- Credit Card = 1

```

new_customer_raw <- data.frame(
  Age = 40,
  Experience = 10,
  Income = 84,
  Family = 2,
  CCAvg = 2,
  Education = factor(2, levels = levels(bank$Education)),
  Mortgage = 0,
  Securities.Account = factor(0, levels = levels(bank$Securities.Account)),
  CD.Account = factor(0, levels = levels(bank$CD.Account)),
  Online = factor(1, levels = levels(bank$Online)),
  CreditCard = factor(1, levels = levels(bank$CreditCard))
)

new_customer_dummy <- predict(dummies, newdata = new_customer_raw)

new_customer_scaled <- predict(preProc, new_customer_dummy)

```

The customer is classified as 0, will not accept the loan.

```

accuracy <- c()

for(k in 1:25){
  pred <- knn(train[, -ncol(train)],
              valid[, -ncol(valid)],
              train$Personal.Loan,
              k = k)

  accuracy[k] <- mean(pred == valid$Personal.Loan)
}

data.frame(k = 1:25, accuracy = accuracy) %>%
  arrange(desc(accuracy)) %>%
  head()

```

```

##    k accuracy
## 1 5    0.9640
## 2 1    0.9630
## 3 3    0.9625
## 4 4    0.9625
## 5 6    0.9615
## 6 7    0.9600

```

Best k = 1

```

best_k <- 1

pred_valid <- knn(train[, -ncol(train)],
                  valid[, -ncol(valid)],
                  train$Personal.Loan,
                  k = best_k)

confusionMatrix(pred_valid, valid$Personal.Loan, positive = "1")

```

```

## Confusion Matrix and Statistics
##
##              Reference
## Prediction    0    1
##              0 1782  48
##              1   26 144
##
##              Accuracy : 0.963
##              95% CI   : (0.9538, 0.9708)
##              No Information Rate : 0.904
##              P-Value [Acc > NIR] : < 2e-16
##
##              Kappa   : 0.7753
##
##              McNemar's Test P-Value : 0.01464
##
##              Sensitivity : 0.7500
##              Specificity : 0.9856

```

```
##          Pos Pred Value : 0.8471
##          Neg Pred Value : 0.9738
##          Prevalence : 0.0960
##          Detection Rate : 0.0720
##          Detection Prevalence : 0.0850
##          Balanced Accuracy : 0.8678
##
##          'Positive' Class : 1
##
```

Validation Confusion Matrix:

- Very high specificity
- Strong sensitivity for a highly imbalanced dataset (9.6% positive class)
- Strong overall predictive performance

```
knn(train[, -ncol(train)],
     new_customer_scaled,
     train$Personal.Loan,
     k = best_k)
```

```
## [1] 0
## Levels: 0 1
```

Result:0 (Will not accept loan)

```
set.seed(1)

trainIndex2 <- createDataPartition(bank_norm$Personal.Loan,
                                   p = 0.5, list = FALSE)

train2 <- bank_norm[trainIndex2, ]
temp <- bank_norm[-trainIndex2, ]

validIndex2 <- createDataPartition(temp$Personal.Loan,
                                   p = 0.6, list = FALSE)

valid2 <- temp[validIndex2, ]
test <- temp[-validIndex2, ]

model_train2 <- knn(train2[, -ncol(train2)],
                    train2[, -ncol(train2)],
                    train2$Personal.Loan,
                    k = best_k)

confusionMatrix(model_train2, train2$Personal.Loan, positive = "1")
```

```
## Confusion Matrix and Statistics
##
##          Reference
## Prediction    0    1
```

```
##          0 2260    0
##          1    0  240
##
##          Accuracy : 1
##          95% CI : (0.9985, 1)
##    No Information Rate : 0.904
##    P-Value [Acc > NIR] : < 2.2e-16
##
##          Kappa : 1
##
##    McNemar's Test P-Value : NA
##
##          Sensitivity : 1.000
##          Specificity : 1.000
##    Pos Pred Value : 1.000
##    Neg Pred Value : 1.000
##          Prevalence : 0.096
##    Detection Rate : 0.096
##    Detection Prevalence : 0.096
##    Balanced Accuracy : 1.000
##
##    'Positive' Class : 1
##
```

Perfect classification indicates overfitting.

Validation Set:

```
pred_valid2 <- knn(train2[, -ncol(train2)],
                   valid2[, -ncol(valid2)],
                   train2$Personal.Loan,
                   k = best_k)

confusionMatrix(pred_valid2, valid2$Personal.Loan, positive = "1")
```

```
## Confusion Matrix and Statistics
##
##          Reference
## Prediction    0    1
##          0 1340   46
##          1   16   98
##
##          Accuracy : 0.9587
##          95% CI : (0.9473, 0.9682)
##    No Information Rate : 0.904
##    P-Value [Acc > NIR] : 9.086e-16
##
##          Kappa : 0.7374
##
##    McNemar's Test P-Value : 0.0002305
##
##          Sensitivity : 0.68056
##          Specificity : 0.98820
##    Pos Pred Value : 0.85965
```

```
##          Neg Pred Value : 0.96681
##          Prevalence : 0.09600
##          Detection Rate : 0.06533
##          Detection Prevalence : 0.07600
##          Balanced Accuracy : 0.83438
##
##          'Positive' Class : 1
##
```

Test Set:

```
pred_test <- knn(train2[, -ncol(train2)],
                 test[, -ncol(test)],
                 train2$Personal.Loan,
                 k = best_k)

confusionMatrix(pred_test, test$Personal.Loan, positive = "1")
```

```
## Confusion Matrix and Statistics
##
##          Reference
## Prediction  0   1
##          0 884  30
##          1  20  66
##
##          Accuracy : 0.95
##          95% CI : (0.9346, 0.9627)
##          No Information Rate : 0.904
##          P-Value [Acc > NIR] : 5.473e-08
##
##          Kappa : 0.6979
##
##          Mcnemar's Test P-Value : 0.2031
##
##          Sensitivity : 0.6875
##          Specificity : 0.9779
##          Pos Pred Value : 0.7674
##          Neg Pred Value : 0.9672
##          Prevalence : 0.0960
##          Detection Rate : 0.0660
##          Detection Prevalence : 0.0860
##          Balanced Accuracy : 0.8327
##
##          'Positive' Class : 1
##
```

- Training set shows perfect accuracy $k = 1$ memorizes data (overfitting).
- Validation and test sets show similar performance.
- Model generalizes reasonably well.
- Low false positives good for reducing marketing cost.
- Some false negatives bank may miss potential borrowers.

Since only 9.6% of customers accept loans:

- Model performs very well at identifying non-acceptors.
- Marketing team should target predicted class 1 customers.

1. $k = 1$ classification 0
2. Best $k = 1$
3. Validation confusion matrix shown above
4. Customer classified as 0
5. Training overfits; validation/test similar acceptable generalization